

CODE SPORT



In this month's column, we feature a medley of computer science interview questions.

A few readers have requested that I continue the computer science interview questions from last month's column. But before I do that, I would like to thank all of you who sent me the solutions to the questions in that column, which will be discussed next month.

As you solve these questions, please do make sure to test your code with normal inputs as well as corner cases. Also, do write down any assumptions you may have regarding the inputs to your code. This will allow you to create test inputs that violate those assumptions, and ensure that your code handles these unexpected inputs gracefully and informatively to the user without crashing. It is also important that you explicitly list the invariants you expect to hold true at different points of your code during coding, and verify that the code you write honours these invariants. One way of doing this programmatically is to use 'assert' statements. Another is to document these invariants in the 'Comments' section and mentally run through the code to ensure that these invariants hold true for all possible inputs.

In some cases, there may be more than one way of solving the same problem. For instance, some problems may have both a recursive and non-recursive solution. Some problems may be solved using either dynamic programming or a recursive approach. While one approach may be efficient over another in terms of space or time complexity, one good practice is to implement both and verify that their results agree for each of your test inputs. This technique is known as using cross-referencing oracles in software testing. This allows automatic checking instead of manually checking results of each of the test inputs.

Here is the set of this month's interview questions.

1. Given an array A of N integers, write a function which returns the index k if there is an array element A[k] such that $A[k] = k$, and which

returns -1 if no such element exists. You can assume that there is no upper limit on the values of elements in A and there are no duplicate elements. What is the time and space complexity of your algorithm? How would your code change if there are duplicate elements in Array A? Does this have any impact on the time and space complexity of your code?

2. Decision trees are one of the most frequently used classifiers in machine learning. Let us assume that you are using a decision-tree classifier to solve a binary classification problem. The input data X contains 10 features/attributes, all of which are categorical. The total number of training samples is 10,000. For the decision-tree classifier, we can use either information gain or gini index to decide which attribute to use for splitting at each node. What is the difference between splitting based on information gain and gini index? Which metric is better for this problem? Suppose you are told that you have 100 features, all of which are categorical. The total number of training samples is 100,000. Instead of binary classification, you are asked to solve a multi-class classification problem. Will your answer change for this problem? If yes, why? If no, why not?
3. Consider a sequence S of letters in English language. This sequence represents a syntactically and semantically valid sentence in English language; however, the white spaces have been removed from the original sentence. You are now asked to segment this sequence to recover the original sentence. Let us assume that there are no other punctuation marks in the original sentence, and words in this sentence are separated by a single white space. You are given an auxiliary blackbox routine 'IsWord'. This routine takes